

# Avaliação: POO

(Classes Abstratas, Métodos Especiais e Duck Typing)

## Seção 1: Múltipla Escolha e Conceitos (5,5 pontos)

### Questão 1 (1,0 ponto)

Qual é a implementação correta e moderna em Python para uma classe abstrata Forma com um método abstrato obrigatório calcular\_area?

A)

```
class Forma:
    def calcular_area(self):
        pass
```

B)

```
from abc import ABC, abstractmethod
class Forma(ABC):
    @abstractmethod
    def calcular_area(self):
        pass
```

C)

```
from abc import abstractmethod
class Forma:
    @abstractmethod
    def calcular_area(self):
        pass
```

D)

```
class Forma(ABC):
    def calcular_area(self):
        raise NotImplementedError("Subclasse deve implementar este método")
```

### Questão 2 (1,0 ponto)

Dado o código:

```
from abc import ABC, abstractmethod
class Veiculo(ABC):
    @abstractmethod
    def ligar(self):
        pass
```

O que acontece ao executar `v = Veiculo()`?

- A) Cria um objeto normalmente, mas `v.ligar()` falha.
- B) `TypeError: Can't instantiate abstract class Veiculo with abstract method ligar`
- C) Executa o método `ligar()` automaticamente na instânciação.
- D) `SyntaxError: Código de classe abstrata inválido.`

### Questão 3 (1,0 ponto)

Dada a classe abstrata:

```
from abc import ABC, abstractmethod
class Notificador(ABC):
    @abstractmethod
    def enviar(self, msg): pass
    @abstractmethod
    def confirmar_leitura(self): pass

class NotificadorEmail(Notificador):
    def enviar(self, msg):
        print(f"Enviando email: {msg}")
```

O que acontece ao tentar executar `n = NotificadorEmail()`?

- A) Cria o objeto normalmente.
- B) `TypeError: Can't instantiate abstract class NotificadorEmail with abstract method confirmar_leitura`
- C) Executa o método `enviar()` automaticamente.
- D) O objeto é criado, mas `n.confirmar_leitura()` retorna `None`.

### Questão 4 (1,5 pontos)

Considere a função que usa Duck Typing:

```
def tocar_midia(lista_de_midias):
    for midia in lista_de_midias:
        midia.play()
```

E as classes:

```
class MP3:
    def play(self): print("Tocando MP3")

class Video:
    def play(self): print("Tocando Vídeo")
```

```
class Radio:
    def sintonizar(self): print("Sintonizando rádio")
```

O que acontece ao chamar `tocar_midia([MP3(), Vídeo(), Radio()])`?

A) Imprime:

```
Tocando MP3
Tocando Vídeo
Sintonizando rádio
```

B) Imprime apenas "Tocando MP3" e "Tocando Vídeo", e ignora Radio silenciosamente.

C) `AttributeError: 'Radio' object has no attribute 'play'`

D) `TypeError: 'Radio' não é do tipo 'MP3' ou 'Vídeo'`

### Questão 5 (1,0 ponto)

Julgue as afirmações abaixo como Verdadeiro (V) ou Falso (F):

- ( ) O Duck Typing em Python exige que as classes herdem de uma interface (Classe Abstrata) comum para serem usadas na mesma função.
- ( ) Uma classe pode ser abstrata (herdar de ABC) e, ao mesmo tempo, implementar métodos especiais como `__len__` ou `__str__`.
- ( ) Se `__str__` não for definido em uma classe, `print(obj)` usará automaticamente o retorno de `__repr__` (se este existir).
- ( ) O método `__init__` é um método especial usado para criar o objeto em memória.

## Seção 2: Implementação e Análise (4,5 pontos)

### Questão 6 (1,5 pontos)

Explique a diferença de propósito e público-alvo entre os métodos especiais `__str__` e `__repr__`.

- `__str__`: \_\_\_\_\_
- `__repr__`: \_\_\_\_\_

### Questão 7 (1,0 ponto)

Implemente o método especial necessário na classe `Vetor2D` para que a operação de soma (+) funcione corretamente.

```
class Vetor2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

# SEU CÓDIGO AQUI

def __repr__(self):
    # __repr__ já fornecido para facilitar a visualização
    return f"Vetor2D({self.x}, {self.y})"

# Teste:
v1 = Vetor2D(1, 2)
v2 = Vetor2D(10, 20)
print(v1 + v2) # Deve resultar em: Vetor2D(11, 22)
```

### Questão 8 (1,0 ponto)

Implemente o método especial na classe Pessoa para que a comparação de igualdade (==) retorne True se, e somente se, o CPF for idêntico. A comparação deve ser segura contra tipos diferentes.

```
class Pessoa:
    def __init__(self, nome, cpf):
        self.nome = nome
        self.cpf = cpf

# SEU CÓDIGO AQUI

# Teste:
p1 = Pessoa('Ana', '123.456')
p2 = Pessoa('Ana Silva', '123.456')
p3 = Pessoa('Bruno', '789.000')
print(p1 == p2) # Deve ser True
print(p1 == p3) # Deve ser False
print(p1 == "123.456") # Deve ser False
```

### Questão 9 (1,0 ponto)

Você está criando uma classe CarrinhoDeCompras que armazena Produtos em uma lista interna self.produtos. A classe deve atender aos seguintes requisitos:

1. len(carrinho) deve retornar o número de produtos.
2. carrinho[0] deve retornar o primeiro produto da lista.

3. for produto in carrinho: deve ser possível iterar sobre os produtos.

Qual método especial (ou métodos) você pode implementar para atender **todos** os três requisitos de forma mais simples? Implemente-o.

```
class CarrinhoDeCompras:  
    def __init__(self):  
        self.produtos = [] # Ex: [Produto('A'), Produto('B')]
```

## Gabarito e Justificativas (Para o Professor)

**Q1: B.** É a forma canônica. A não é abstrata. C falta herdar de ABC. D usa `NotImplementedError`, que é uma alternativa válida, mas não *impede* a instancição como ABC faz, sendo menos rigoroso.

**Q2: B.** Classes abstratas com métodos abstratos não implementados não podem ser instanciadas diretamente.

**Q3: B.** Uma subclasse de uma classe abstrata deve implementar **todos** os métodos abstratos da classe pai para poder ser instanciada. Faltou confirmar `_leitura`.

**Q4: C.** Duck Typing: O Python tenta chamar `.play()` em todos os objetos. MP3 e Video funcionam. Radio não tem o método `.play()`, resultando em um `AttributeError`. A alternativa D está errada pois o Python se *importa* com o tipo em tempo de execução se o método não existir.

**Q5: F, V, V, F**

- (F) Duck Typing é o *oposto* disso. Importa o comportamento (ter o método), não a herança.
- (V) Classes abstratas são classes e podem ter qualquer outro método, concreto ou especial.
- (V) Este é o comportamento "fallback" padrão de `print()`.
- (F) `__init__` *inicializa* um objeto que já foi criado. `__new__` é o método responsável pela *criação* (alocação).

**Q6:**

- `__str__`: Define a representação "amigável" do objeto, focada no **usuário final**. É o que `print(obj)` e `str(obj)` usam. (Ex: "Produto: Camisa, R\$ 50,00")
- `__repr__`: Define a representação "oficial" ou "de desenvolvedor" do objeto. O ideal é que seja um código Python válido que possa recriar o objeto. É focado no **desenvolvedor** (para debugging, logs). (Ex: `Produto('Camisa', 50.00)`)

**Q7:**

```
def __add__(self, outro):  
    # Idealmente, verificar se 'outro' é Vetor2D  
    if not isinstance(outro, Vetor2D):  
        return NotImplemented  
    return Vetor2D(self.x + outro.x, self.y + outro.y)
```

**Q8:**

```
def __eq__(self, outro):  
    # 1. Verificar o tipo para evitar AttributeError  
    if not isinstance(outro, Pessoa):  
        return False  
    # 2. Comparar o atributo relevante  
    return self.cpf == outro.cpf
```

**Q9:** `__getitem__`

```
def __getitem__(self, indice):  
    return self.produtos[indice]
```

- **Justificativa:** Ao implementar `__getitem__`, o Python automaticamente ganha suporte a acesso por índice (`carrinho[0]`), iteração (`for...in`) e `len()` (desde que `__len__` não esteja explicitamente definido, ele usará a iteração para contar). Implementar `__len__` e `__iter__` separadamente também estaria correto, mas `__getitem__` é a resposta mais simples que resolve os três.